Artificial Intelligence | Healthcare Technology | FinTech | Cybersecurity | Social Impact

# ClaimGuard AI Technical Documentation

## AI Health-Insurance Claims Adjudication & Fraud Defence

A Multi-Agent AI Decision Layer for Faster, Fairer & Fraud-Resistant Health Claims

---

**Version** 1.0.0 · **Status** MVP Simulation (Hackathon Prototype)

**Stack** FastAPI · SQLite · React · TypeScript · Tailwind · Recharts

**Scope** Full-stack working prototype — 14 UI pages, 21 REST endpoints, 35 automated tests

**Disclaimer:** Explainable rule-based demo logic over anonymized mock data. No real PM-JAY / NHCX integration, no real patient data, no certified medical decisioning.

# Table of Contents

# 1 · Executive Summary

ClaimGuard AI is a multi-agent AI platform that acts as an intelligent adjudication and fraud-defence layer on top of health-insurance claim workflows (hospital portals, insurers, TPAs, PM-JAY and the NHCX exchange).

It receives claim data and documents, extracts and normalizes information, runs ten specialized verification agents, calculates a 0–100 fraud / medical risk score, generates an explainable recommendation, allows human-in-the-loop auditor action, logs every decision to an audit trail, and feeds auditor feedback into a simulated learning loop.

## What is delivered in this prototype

| | |
|---|---|
| **14** FRONTEND PAGES / ROUTES | **12** SIGNALS IN RISK CATALOGUE |
| **21** REST API ENDPOINTS | **12** SEEDED SCENARIO CLAIMS |
| **10** VERIFICATION AGENTS | **35** AUTOMATED TESTS (PASSING) |

## Core capabilities

- **Deterministic, explainable risk engine** — a fixed, auditable 12-signal catalogue, weighted-sum scoring capped at 100, mapped to four action bands.
- **Ten-agent verification pipeline** with per-agent status, confidence, evidence and risk contribution.
- **Explainable decision report** with an exact score-attribution waterfall, evidence table, audit trail, print / JSON export.
- **Human-in-the-loop adjudication** — five auditor actions, each producing an audit-log entry and a feedback event.
- **Forensic analytics lab** — Benford's-Law billing test, provider peer-outlier z-scores, ghost-beneficiary / identity-cluster detection and a collusion network graph.
- **Policy analytics, fraud workspace, hospital portal, audit logs, feedback loop, architecture & demo guide.**

**MVP Simulation.** Every clinical, billing, fraud and identity check is explainable demo logic, not certified adjudication. All data is anonymized mock data ( `BEN-xxxx` , `HOSP-xxx` , `CLM-xxxx` ). The system is designed so a human auditor always makes the final decision.

# 2 · Problem Statement

India's health-insurance systems process a massive volume of claims with hospital bills, discharge summaries, diagnostic reports, treatment codes, patient identity, medical images and supporting documents. Many claims are genuine, but a meaningful fraction contain anomalies or fraud.

**~40,000**
DAILY CLAIMS UNDER PM-JAY

**9.84 Cr**
HOSPITAL ADMISSIONS AUTHORIZED

**₹1.40L Cr**
VALUE PROCESSED

**1,900+**
TREATMENT PACKAGES

Context figures from the source PDF; shown for framing only — this prototype uses no live PM-JAY data.

## Fraud & failure typologies (hard to detect manually)

| Pattern | Example | Why hard to detect manually |
|---|---|---|
| Document manipulation | Edited PDF, altered watermark, fake report | Looks visually similar to genuine documents |
| Package mismatch | Claim filed under a higher-value package | Needs medical + billing + policy knowledge |
| Duplicate procedures | Same procedure claimed multiple times | Needs cross-claim history analysis |
| Repeat admission fraud | Same patient repeatedly admitted for a similar issue | Needs longitudinal patient-level tracking |
| Reused images | Same diagnostic image used in multiple claims | Needs image hashing and similarity matching |
| Inflated billing | Consumables, room charges, implants over-billed | Needs package-rate and policy-rule comparison |
| Ghost identity | Fake or ineligible beneficiary used | Needs identity + eligibility + pattern checks |
| AI-generated forgery | Fabricated clinical narrative via AI tools | Needs LLM-based consistency and forgery signals |

## Why manual review is not enough

- **Delay** — every claim waits for human reading.
- **Inconsistency** — different auditors reach different judgments.
- **Reactive** — fraud is detected after payment.
- **Scale** — manual review cannot handle 40,000+ claims/day.

# 3 · Solution Overview & Decision Outputs

ClaimGuard AI is an AI co-pilot that auto-clears low-risk claims, flags suspicious ones, gives auditors a one-page risk summary, creates an audit trail, and learns from feedback — keeping a human in the loop for every decision. Claims pass through specialized agents → combined risk score → explainable recommendation.

## Decision outputs

| Output | Meaning |
|---|---|
| Claim Score | 0–100 score showing claim trustworthiness |
| Fraud Risk | Low / Medium / High / Critical |
| Medical Necessity | Supported / Weak / Not Supported |
| Document Completeness | Complete / Missing / Suspicious |
| STG Compliance | Follows guideline / Partial / Violates |
| Recommended Action | Auto-approve / Query / Human audit / Fraud hold |
| Explainability | Human-readable reasons and evidence |

## Risk bands & routing

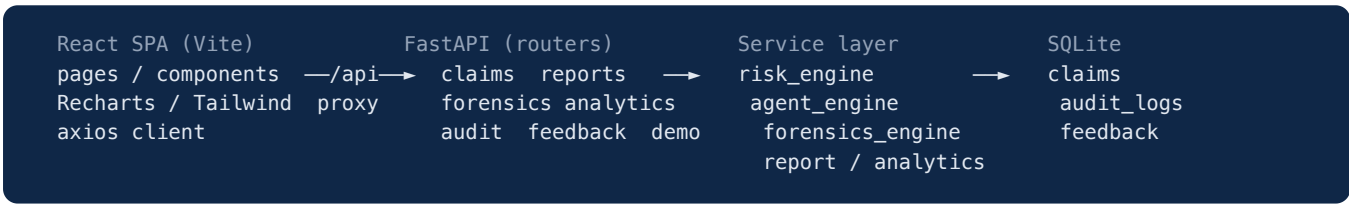
| Score | Category | Recommended action | Description |
|---|---|---|---|
| 0–25 | Low Risk | Auto-approve / Fast-track | Complete documents, guideline compliant, no fraud flags |
| 26–50 | Medium Risk | Query Hospital | Missing or unclear evidence, needs clarification |
| 51–75 | High Risk | Medical Auditor Review | Anomaly detected, document or image issue |
| 76–100 | Critical Risk | Fraud Investigation Hold | Identity / document fraud indicators present |

## What makes the solution unique

- **Multi-agent** — each agent specializes in one verification domain rather than one generic model.
- **Combined reasoning** — medical + document forensics + billing audit + anomaly detection.
- **Human-in-the-loop** — designed for safe AI-assisted review, not unsafe full automation.
- **NHCX-compatible by design** — a normalization agent targets India's standardized claim exchange.
- **Explainable** — every recommendation carries evidence and a traceable score breakdown.

# 4 · System Architecture

A decoupled single-page application talks to a FastAPI REST backend; the backend's service layer implements the AI decision logic over a SQLite store.




## Three-layer design (from the source PDF)

| Layer | Components | Status in MVP |
|---|---|---|
| **1 · Integration** | Hospital portal, Insurer system, PM-JAY, TPA, NHCX gateway | Represented on the Architecture page; **not live** (roadmap) |
| **2 · AI Decision** | API gateway, claim processing, document intelligence, multi-agent orchestrator, rules engine, vector knowledge base, ML model services → Risk Score + Explainable Recommendation | Orchestrator + rules engine **implemented** (deterministic); ML services are placeholders |
| **3 · User** | Auditor dashboard, hospital dashboard, fraud investigation, policy analytics | **Implemented** |
| **Security (vertical)** | RBAC, OAuth2, AES-256, immutable audit trails, model-output logging, human override, bias monitoring, consent-aware usage | Audit trail, role selection, model-output logging, human override **implemented**; OAuth2 / AES-256 roadmap |

Components labelled **roadmap** are explicitly not live in this prototype and are shown for architectural completeness only.

# 5 · Technology Stack

| Tier | Technology | Role in this prototype |
|---|---|---|
| Frontend | React 18, Vite 6, TypeScript 5 | SPA, type-safe API contract, fast dev/build |
| UI | Tailwind CSS 3, Lucide React | White/navy/teal enterprise theme, icons |
| Charts | Recharts 2 | Risk distribution, Benford, peer-outlier radar, scorecards |
| Routing/HTTP | React Router 6, Axios | 14 routes, typed API client via `/api` proxy |
| Backend | FastAPI, Uvicorn | Async REST API, OpenAPI docs at `/docs` |
| Validation | Pydantic v2 | Request/response schemas |
| Persistence | SQLAlchemy 2, SQLite | 3 tables, single-file DB, auto-seed |
| Testing | Pytest, HTTPX TestClient | 35 tests across engines + endpoints |
| Packaging | Docker, docker-compose | Optional containerized run |

The PDF's recommended production stack additionally cites LangGraph/CrewAI orchestration, LLM/OCR/ML model services, Postgres + MongoDB + vector DB, and FHIR/ICD-10/SNOMED data standards — these are roadmap targets, not part of the MVP.

# 6 · Repository Structure

```
claimguard-ai/
├── backend/
│   ├── app/
│   │   ├── main.py            # FastAPI app, CORS, lifespan auto-seed, /health
│   │   ├── database.py        # SQLite engine + session
│   │   ├── models.py          # Claim / AuditLog / Feedback ORM
│   │   ├── schemas.py         # Pydantic request/response models
│   │   ├── seed_data.py       # 12 anonymized scenario claims
│   │   ├── services/
│   │   │   ├── risk_engine.py       # 12-signal weighted scoring, bands
│   │   │   ├── agent_engine.py      # 10-agent verification + AI summary
│   │   │   ├── report_engine.py     # explainable report JSON
│   │   │   ├── analytics_engine.py  # distributions, scorecards
│   │   │   └── forensics_engine.py  # Benford, z-scores, identity, network, attribution
│   │   └── routers/  # claims, reports, forensics, analytics, audit, feedback, demo
│   ├── tests/        # health, claims, risk_engine, reports, forensics
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── pages/        # 14 route pages
│   │   ├── components/   # Layout, RiskGauge, AgentCard, ScoreWaterfall, ui…
│   │   ├── api/client.ts # typed axios client
│   │   ├── types/ data/ utils/ styles/
│   └── package.json
├── docs/   # SOURCE_ALIGNMENT, API, TECHNICAL, DEMO_SCRIPT, TEST_REPORT, LIMITATIONS
├── docker-compose.yml
└── README.md
```

# 7 · Data Model

Three SQLite tables (SQLAlchemy ORM). All identifiers are anonymized mock values.

## claims

| Column | Type | Notes |
|---|---|---|
| claim_id | str (unique) | e.g. `CLM-2048-AYU` |
| beneficiary_id / hospital_name / hospital_id | str | anonymized |
| admission_date / discharge_date / submission_date | str | ISO dates |
| diagnosis / procedure_code / treatment_package | str | clinical fields |
| claim_amount / claimed_package_rate | float | billing |
| documents | JSON | 6-item checklist of booleans |
| signals | JSON | 12 risk-signal booleans |
| ocr_text | text | optional simulated OCR input |
| risk_score / risk_category / recommended_action | int / str / str | computed |
| fraud_risk / medical_necessity / document_completeness / stg_compliance | str | derived statuses |
| ai_summary / status / created_at | text / str / datetime | computed + workflow |

## audit_logs

claim_id · action · user_role · previous_status · new_status · note · reason · evidence_id · output_hash (placeholder) · timestamp.

## feedback

claim_id · feedback_type (confirmed_fraud / false_positive / false_negative / true_positive) · auditor_decision · ai_recommendation · note · user_role · timestamp.

# 8 · Risk Scoring Engine

`services/risk_engine.py` implements deterministic, fully auditable scoring. The score is the capped sum of the weights of all active signals.

```
score = min( Σ weight(signal) for signal in active_signals , 100 )
```

## Signal catalogue & weights

| Signal key | Label | Category | Weight | Owning agent |
|---|---|---|---|---|
| identity_issue | Identity / eligibility issue | Identity | **25** | Identity & Eligibility |
| reused_image | Reused image signal | Image | **22** | Image Integrity |
| suspicious_document | Suspicious document authenticity | Document | **20** | OCR & Document |
| package_mismatch | Treatment package mismatch | Billing | **18** | Billing Audit |
| duplicate_procedure | Duplicate procedure | Fraud | **18** | Fraud Pattern |
| stg_non_compliance | STG non-compliance | Medical | **18** | STG Compliance |
| diagnosis_treatment_mismatch | Diagnosis-treatment mismatch | Medical | **16** | Medical Coding |
| repeat_admission | Repeat admission pattern | Fraud | **15** | Fraud Pattern |
| ai_generated_note | AI-generated clinical note suspicion | Document | **15** | OCR & Document |
| inflated_billing | Inflated billing | Billing | **14** | Billing Audit |
| missing_document | Missing mandatory document | Document | **12** | OCR & Document |
| high_risk_hospital | High-risk hospital pattern | Fraud | **10** | Fraud Pattern |

## Derived signals (computed from data, not just toggles)

- **missing_document** is set when any mandatory document (hospital bill, discharge summary, diagnostic report, patient identity, admission proof) is absent.
- **inflated_billing** is set when `claim_amount > claimed_package_rate × 1.15`.

## Derived qualitative statuses

| Output | Rule |
|---|---|
| Document Completeness | Suspicious if identity/suspicious-doc/reused-image; else Missing if missing-doc; else Complete |
| Medical Necessity | Not Supported if diagnosis-mismatch/package-mismatch; else Weak if STG non-compliance; else Supported |
| STG Compliance | Violates if STG non-compliance; else Partial if diagnosis/package mismatch; else Follows guideline |

The engine also returns the full per-signal factor breakdown and the top-5 contributors, which drive the report and the explainability views.

# 9 · Multi-Agent Verification Engine

`services/agent_engine.py` runs ten agents in a fixed pipeline. Each agent owns a subset of signals and reports **status**, **confidence**, **input checked**, **output**, **evidence**, **risk contribution** and **explanation**.

### STATUS LOGIC

```
any owned signal with weight ≥ 18 active   → Failed
any owned signal active                    → Warning
otherwise                                  → Passed
```

| # | Agent | Verifies | Owned signals |
|---|---|---|---|
| 1 | Claim Intake | Mandatory fields present & valid | field validation |
| 2 | OCR & Document | Document completeness & authenticity | missing_document, suspicious_document, ai_generated_note |
| 3 | FHIR/NHCX Normalization | Normalizes to NHCX-compatible structure | (warns if procedure code missing) |
| 4 | Identity & Eligibility | Beneficiary identity / scheme eligibility | identity_issue |
| 5 | Medical Coding | ICD / procedure vs diagnosis | diagnosis_treatment_mismatch |
| 6 | STG Compliance | Treatment vs Standard Treatment Guidelines | stg_non_compliance |
| 7 | Billing Audit | Inflation & package-rate mismatch | inflated_billing, package_mismatch |
| 8 | Image Integrity | Reused / tampered images | reused_image |
| 9 | Fraud Pattern | Duplicates, repeat admissions, hospital anomaly | duplicate_procedure, repeat_admission, high_risk_hospital |
| 10 | Explainability | Synthesizes rationale + recommendation | combined output |

Confidence is a deterministic function of agent state (no randomness), so runs are reproducible. The engine also produces the one-paragraph AI claim summary used across the UI.

# 10 · Forensic Analytics Engine

`services/forensics_engine.py` adds statistics-based fraud forensics. Each technique is a **real** fraud-analytics method applied to anonymized mock data.

### 10.1 Benford's Law billing analyzer

Expands claims into a deterministic billing ledger, computes the first-digit distribution versus the Benford expectation `log10(1 + 1/d)`, and the Mean Absolute Deviation (MAD) against Nigrini conformity bands.

| MAD range | Conformity band |
|---|---|
| < 0.006 | Close conformity |
| 0.006–0.012 | Acceptable conformity |
| 0.012–0.015 | Marginal conformity |
| ≥ 0.015 | **Nonconformity** — possible fabricated amounts |

### 10.2 Provider peer-outlier radar

Computes per-hospital metric means and standard deviations, converts to z-scores (avg claim value, signal density, risk score); a peak $|z| \geq 1.5$ marks a provider as an outlier.

### 10.3 Ghost-beneficiary / identity-cluster detector

Replicates patterns from real CAG audits of PM-JAY using synthetic records: beneficiaries sharing a single mobile number, claims on patients recorded as deceased, and simultaneous double admissions.

### 10.4 Collusion network graph

Builds a graph of hospital ↔ beneficiary ↔ broker links plus shared-phone edges, applies a deterministic circular layout, and scores rings by member count.

### 10.5 Score-attribution waterfall

Because scoring is rule-based, attribution is **exact** (not estimated SHAP): every point of the 0–100 score is traced to a named signal, presented as a cumulative waterfall from a clean baseline.

### Grounding sources

| Technique | Source |
|---|---|
| Benford MAD thresholds | Nigrini conformity method (NCBI PMC4809611) |
| Peer-outlier z-scores | Medicare / dermatology payment audits (PMC12193562) |
| Ghost beneficiary / shared phone | CAG findings on PM-JAY (7.5 lakh beneficiaries on one number; claims on deceased) |
| Collusion graph | Graph-based FWA detection (AAAI) |
| Score attribution | SHAP-style additive attribution for fraud XAI |

**Honesty guardrail.** These techniques are applied to mock data. The identity detector replicates the *pattern* of real findings on synthetic records — it uses no real beneficiary data. Production would require full history, case-mix adjustment, live NHCX data and validated ML.

# 11 · Report & Analytics Engines

## Report engine ( `report_engine.py` )

Assembles claim + agent results + audit history into a structured report:

- Claim trust score (100 – risk), risk score, fraud risk, medical necessity, document completeness, STG compliance.
- Evidence table (signal → category → detecting agent → contribution → detail).
- Top-5 risk reasons, auditor notes, full audit trail.
- Model/rule explanation (raw total → capped score), output-hash placeholder.
- Human-in-the-loop safety note and the MVP-Simulation disclaimer.

## Analytics engine ( `analytics_engine.py` )

| Output | Computation |
|---|---|
| Risk & status distribution | Counts over all claims |
| Fast-track rate | Share of Low-Risk (auto-approve) claims |
| Avg review time | Band-weighted simulated minutes |
| High-risk packages | Average risk per treatment package |
| Hospital scorecards | Per-hospital avg risk, flagged count, signal total, anomaly level |
| Fraud-pattern counts | Frequency of each signal across the portfolio |
| Feedback summary | Confirmed-fraud / false-positive / false-negative / true-positive tallies |

# 12 · End-to-End Claim Workflow

The PDF's 10-step pipeline, as implemented in the prototype:

| Step | Stage | Implementation |
|---|---|---|
| 1 | Hospital claim submission | `POST /claims` (Claim Intake page) |
| 2 | Data ingestion API | Pydantic validation → persisted claim |
| 3 | OCR + document parsing | Document checklist + optional OCR text (simulated) |
| 4 | FHIR/NHCX normalization | Normalization agent (simulated bundle) |
| 5 | Multi-agent verification | `agent_engine.run_agents` (10 agents) |
| 6 | Fraud & medical risk score | `risk_engine.score_claim` (0–100, banded) |
| 7 | Explainable recommendation | Report engine + score-attribution waterfall |
| 8 | Auto-approve / query / audit / fraud-hold | Band → recommended action; auditor confirms |
| 6b / 9 | Human-in-the-loop review & settlement | `POST /claims/{id}/decision` → status + audit |
| 10 | Feedback learning loop | Feedback events + simulated rule/drift queue |

# 13 · Backend API Reference

Base URL `http://localhost:8000`; frontend calls via the Vite `/api` proxy. Interactive docs at `/docs`. All responses are JSON. MVP — no auth.

### System & claims

| Method | Path | Purpose |
|---|---|---|
| GET | /health | Service health + mode |
| GET | /claims | List + filter (risk_category, status, hospital, package, search) |
| GET | /claims/{id} | Single claim |
| POST | /claims | Create + analyze (auto claim_id if omitted) |
| POST | /claims/{id}/analyze | Re-run risk engine + agents |
| GET | /claims/{id}/agents | 10 agent result cards + combined score |
| POST | /claims/{id}/decision | Record auditor decision → status + audit + feedback |

### Reports & forensics

| Method | Path | Purpose |
|---|---|---|
| GET | /claims/{id}/report | Explainable decision report |
| GET | /claims/{id}/explanation | Deterministic score-attribution waterfall |
| GET | /forensics/benford?hospital= | Benford first-digit test + MAD band |
| GET | /forensics/peer-outliers | Provider z-score outliers |
| GET | /forensics/identity-flags | Shared-phone clusters, deceased claims, double admissions |
| GET | /forensics/network | Collusion network nodes/edges/rings |

### Analytics, audit, feedback, demo

| Method | Path | Purpose |
|---|---|---|
| GET | /analytics/overview | KPIs + distributions + package risk |
| GET | /analytics/fraud-patterns | Signal counts + fraud queues |
| GET | /analytics/hospitals | Hospital anomaly scorecards |
| GET | /audit-logs?claim_id= | Audit trail |
| GET / POST | /feedback | Feedback summary + events / create event |
| GET / POST | /demo/seed · /demo/reset | (Re)seed the 12 scenario claims |

#### EXAMPLE — POST /CLAIMS/{ID}/DECISION

```
// request
{ "action": "Fraud Hold", "note": "Six fraud indicators.", "user_role": "Fraud Investigator" }
```

```
// response
{ "claim": { "status": "Fraud Hold", ... }, "audit_log": { "action": "Fraud Hold", ... } }
```

Allowed actions: Approve · Query Hospital · Send to Medical Audit · Fraud Hold · Reject.

# 14 · Frontend Architecture

React 18 + TypeScript + Vite SPA. A typed axios client ( `api/client.ts` ) mirrors the backend schemas ( `types/index.ts` ). A small `useAsync` hook standardizes loading / error / refetch. Charts use `isAnimationActive=false` for instant, demo-stable rendering.

## Design system

- **Theme** — white background, navy text ( `#0f2747` ), teal/blue accents ( `#0d9488` ), soft cards, thin borders, rounded corners.
- **Risk colors** — Low green, Medium amber, High orange, Critical red.
- **Every page** carries a "What this page demonstrates" intro and an `MVP Simulation` badge.

## Reusable components

Layout (sidebar + topbar) · StatCard · RiskBadge · StatusBadge · ClaimTable · RiskGauge · AgentCard · EvidenceTable · AuditTimeline · ScoreWaterfall · ChartCard · LoadingState · EmptyState · ErrorState · Modal · MvpBadge · PageIntro · ConfidenceBar.

## Routing & state

React Router 6 with 14 routes (future flags enabled). Data fetched per-page via the `useAsync` hook against the typed client; the Vite dev server proxies `/api/*` → `http://localhost:8000` , so no CORS friction in development (backend CORS is also open for the MVP).

# 15 · Page-by-Page Walkthrough

| Route | Page | What it demonstrates |
|---|---|---|
| / | Landing / Overview | Problem, value props, live KPI cards, CTAs |
| /claims | Claims Dashboard | Filterable/searchable queue with live risk scores |
| /claims/new | Claim Intake | Hospital submission with a live risk preview + signal toggles |
| /claims/:id | Claim Review | AI summary, gauge, decision outputs, 10 agents, top factors, decision modal, audit trail |
| /agents/:claimId | Multi-Agent Verification | Animated "Run Verification" → combined score |
| /claims/:id/report | Decision Report | Trust score, waterfall, evidence, print / export JSON / copy |
| /analytics | Analytics Dashboard | Risk/status distribution, packages, hospital leaderboard |
| /fraud | Fraud Workspace | Critical queue, per-signal panels, case evidence |
| /forensics | Forensic Analytics Lab | Benford, peer outliers, identity clusters, collusion graph |
| /hospital | Hospital Portal | Claim tracking, queries, missing-doc requests, resubmit |
| /audit | Audit Logs | Full traceable action trail with output hashes |
| /feedback | Feedback Learning Loop | Fraud confirmations, rule-improvement & drift queues (simulated) |
| /architecture | Architecture | Three-layer design + security shield |
| /demo-guide | Demo Guide | Click-by-click 5-minute script with narration |

# 16 · Security Model

| Control | Status | Detail |
|---|---|---|
| Role-based access (RBAC) | Illustrative | Role selected on each decision; not enforced auth |
| Audit trail | Implemented | Every action logged; "immutability" simulated (SQLite rows) |
| Model/rule output logging | Implemented | Output-hash placeholder per decision |
| Human override | Implemented | AI never auto-settles; auditor decides |
| OAuth2 authentication | Roadmap | Not implemented |
| AES-256 encryption at rest | Roadmap | Not implemented |
| Bias / false-positive monitoring | Simulated | Drift queue on the feedback page |
| Consent-aware data usage | Roadmap | Mock data only; no real PII |

# 17 · Scalability & Performance

- **Pure-function engines.** Risk, agent and forensic logic are stateless functions over claim dicts — trivially parallelizable and cacheable.
- **Async API.** FastAPI + Uvicorn is async-ready and horizontally scalable behind a gateway.
- **Deterministic latency.** Scoring and agent runs are O(signals) with no model inference, so per-claim latency is sub-millisecond in the MVP.
- **Production path.** SQLite → PostgreSQL (claims) + document store + dedicated audit-log service; rules engine fronted by a vector knowledge base for policy/STG lookups; optional ML model services (LLM, fraud ML, image forensics) per the PDF.

### PDF MVP target metrics (aspirational)

| Metric | Target |
|---|---|
| Low-risk fast-track rate | 50–70% |
| Reduction in manual review load | 40–60% |
| Document extraction accuracy | 90%+ |
| Fraud recall (high-risk) | 80%+ |
| False hold rate (genuine) | below 5–8% |
| Claim summary generation | under 60s |
| Audit trail coverage | 100% |

These are project targets from the PDF, not measured results of this prototype.

# 18 · Testing & Quality Assurance

## Automated tests — 35 passing

| Suite | Covers |
|---|---|
| test_health | /health and root metadata |
| test_risk_engine | band boundaries (25/26, 50/51, 75/76), score cap at 100, derived signals, top-factor ordering, derived statuses, 10-agent output |
| test_claims | list/filter/search, create+analyze, agents, decision → status + audit, invalid action |
| test_reports | report structure, trust score, analytics overview, fraud patterns, hospital scorecards, feedback, demo reset |
| test_forensics | Benford digits/MAD/band, peer outliers, identity clusters, network graph, exact score attribution |

## Verification performed

- **Backend:** 35 pytest tests pass; all 16 endpoints return HTTP 200; 12 seed claims span all four risk bands.
- **Frontend:** `npm run build` type-checks and builds with zero errors; all 14 routes render; console clean.
- **Browser (Chrome DevTools):** Landing, Claims, Submit, Review (10 agents + gauge + audit), Analytics, Fraud, Forensics, and Report (waterfall) verified rendering with live backend data.

## Seeded scenario claims & scores

| Claim | Scenario | Score | Band |
|---|---|---|---|
| CLM-2001-GEN | Clean low-risk | 0 | Low |
| CLM-2002-NEPHRO | Missing document | 12 | Low |
| CLM-2006-GEN | Repeat admission | 27 | Medium |
| CLM-2004-ORTHO | Inflated billing | 32 | Medium |
| CLM-2003-CARD | Package mismatch | 34 | Medium |
| CLM-2011-DIAB | STG partial compliance | 34 | Medium |
| CLM-2010-NEURO | AI-generated note suspicion | 35 | Medium |
| CLM-2005-OPHTH | Duplicate procedure | 43 | Medium |
| CLM-2008-GASTRO | Suspicious document | 51 | High |
| CLM-2007-RAD | Reused image | 52 | High |
| CLM-2009-MAT | Identity / ghost beneficiary | 82 | Critical |
| CLM-2012-CRIT | Multi-signal fraud | 100 | Critical |

# 19 · Source Alignment (PDF Traceability)

Every feature traces to a concept in the source PDF; nothing adds capabilities the PDF does not describe.

| Feature | PDF concept | Simulated part |
|---|---|---|
| 10-agent verification | Multi-Agent AI Approach | Agent intelligence is rule-mapped (no LLM/ML) |
| 0–100 score + 4 bands | Claim Risk Score Categories | Fixed demo weights |
| Decision outputs | Proposed Solution: Decision Outputs | Rule-derived qualitative statuses |
| Explainable report | Explainable Decision Report | Output hash is a placeholder |
| Human-in-the-loop actions | Human-in-the-Loop Review | — |
| Audit trail | Immutable Audit Trails | "Immutable" simulated (SQLite) |
| Feedback learning loop | Feedback Learning Loop / Step 10 | No model retrained |
| Analytics & scorecards | Policy / Hospital Analytics | Review-time figures simulated |
| Architecture (3 layers) | System Architecture Diagram | Integration/ML layers are roadmap |
| Forensic analytics | Fraud detection deep-dive | Real techniques on mock data |

# 20 · Limitations & Anti-Hallucination Policy

**Must not be claimed.** No real PM-JAY integration · no real NHCX integration · no real hospital deployment · no real patient data · no real/certified fraud accuracy or medical certification · no production-grade medical decisioning · no real OCR / image hashing / ML / model retraining.

## Deliberate limitations

- **Data:** anonymized mock data only; 12 illustrative seed claims.
- **AI:** rule-based scoring (not ML); deterministic templates for summaries; no real LLM.
- **OCR / images:** document checklist + optional pasted text; reused-image is a simulated signal.
- **Score attribution** is exact (rule-based), explicitly *not* estimated SHAP.
- **Fraud / medical:** figures shown anywhere are simulated targets; nothing should be used to approve, deny or settle a real claim.
- **Security:** audit "immutability" simulated; output hashes are placeholders; RBAC illustrative; no real auth/encryption.
- **Feedback loop** captures decisions and surfaces a simulated rule/drift queue; no model is retrained.
- **Persistence:** single-file SQLite suitable for a local demo, not production volume.

# 21 · Demo Script & Setup

## Run locally

```
# Backend (:8000)
cd backend && python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# Frontend (:5173)
cd frontend && npm install && npm run dev
# open http://localhost:5173 — Vite proxies /api to the backend
```

## 5-minute demo flow

| Time | Page | Action / narration |
|---|---|---|
| 0:00 | Landing | "A multi-agent AI decision layer for faster, fairer, fraud-resistant claims." |
| 0:30 | Claims Dashboard | Filter Critical Risk — "Every claim scored 0–100 and triaged." |
| 1:00 | Submit Claim | Toggle signals — live score updates with backend weights. |
| 1:45 | Agent Runner | Run Verification — ten agents report in sequence. |
| 2:30 | Claim Review | Gauge at 100, decision outputs, top factors. |
| 3:15 | Decision modal | Fraud Hold + note — audit log + feedback created. |
| 3:45 | Report | Score-attribution waterfall, export JSON. |
| 4:15 | Forensics + Analytics | Benford nonconformity, shared-phone ring, hospital scorecards. |
| 4:45 | Architecture | Three layers + security shield; NHCX-ready. |
| 5:00 | Close | "Speed for genuine claims, defence against fraud, transparency for all." |

# 22 · Roadmap

| Phase | Production enhancement |
|---|---|
| 1 | NHCX / FHIR-compatible integration |
| 2 | Advanced fraud ML with hospital-patient graph analytics |
| 3 | Image tampering & reuse detection (perceptual hashing, CLIP/CNN) |
| 4 | STG compliance engine across more specialties |
| 5 | Real-time fraud heatmaps and hospital scorecards |
| 6 | Model governance, bias monitoring, audit compliance |
| 7 | Scalable deployment for state/national claim volume |
| Ongoing | Continuous learning from auditor feedback |

**The impact triangle:** Faster Claims + Lower Fraud + Higher Trust = Better Healthcare Access. Even a 20–30% reduction in manual review at national scale means thousands of claims processed faster daily, with funds staying available for genuine patients.

# 23 · Appendix — Glossary

| Term | Meaning |
|---|---|
| PM-JAY / ABPM-JAY | Ayushman Bharat Pradhan Mantri Jan Arogya Yojana — India's national health-insurance scheme |
| NHCX | National Health Claims Exchange — India's standardized, FHIR-based health-claims exchange |
| FHIR | Fast Healthcare Interoperability Resources — health-data exchange standard |
| TPA | Third-Party Administrator — processes claims on behalf of insurers |
| STG | Standard Treatment Guidelines |
| HBP | Health Benefit Package (PM-JAY treatment package) |
| MAD | Mean Absolute Deviation — Benford's-Law conformity measure |
| FWA | Fraud, Waste & Abuse |
| CAG | Comptroller and Auditor General of India |
| SHAP | SHapley Additive exPlanations — feature-attribution method (referenced; our attribution is exact rule-based) |
| RBAC | Role-Based Access Control |